```
          cout << dNo1;
          else
          cout << dNo2;
        cout << " is the greatest";
        break;
      case 2:
        if (dNo1<dNo2)
          cout << dNo1;
          else
          cout << dNo2;
        cout << " is the least";
        break;
      case 3:
        cout << "The average is " << (dNo1+dNo2)/2;
        break;
      default:
        cout << "Wrong choice";
        break;
    }
}
```

The header file stdlib.h is needed to be able to clean the screen with system("cls"), which is done after the user has entered the two values. Then we print the menu on the screen and the user enters his choice (1, 2 eller 3) to the variable iOpt.

The switch statement will then check the variable iOpt. If it is 1, the statements after "case 1" are executed. There we check which of the two numbers are the greatest and print it. In the same way the least number is printed under "case 2". In case of 3, the average is calculated and printed. If the user has entered anything else, the default statements are executed.

3.12 Loops

We will now continue with another powerful tool within programming, that can make the program perform a series of operations a specific number of times. Sometimes, the number of times, or the number of iterations, decided from start, sometimes it depends on the circumstances. We begin with an example:

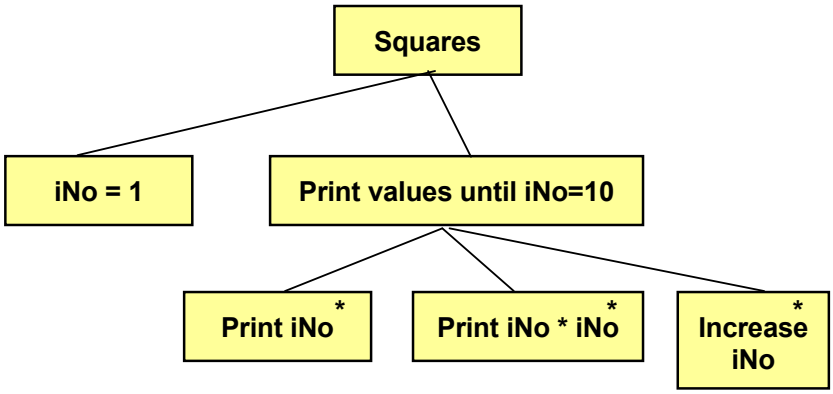
We want to print a list of the numbers 1-10 and their squares:

1 1
2 4
3 9
etc.

We have a variable, iNo, which first has the value 1. We print it and the square of it. Then we increase the value of iNo by 1 and repeat the process, i.e. we print iNo and the square of iNo. Then we increase iNo again etc. This goes on until iNo = 10.

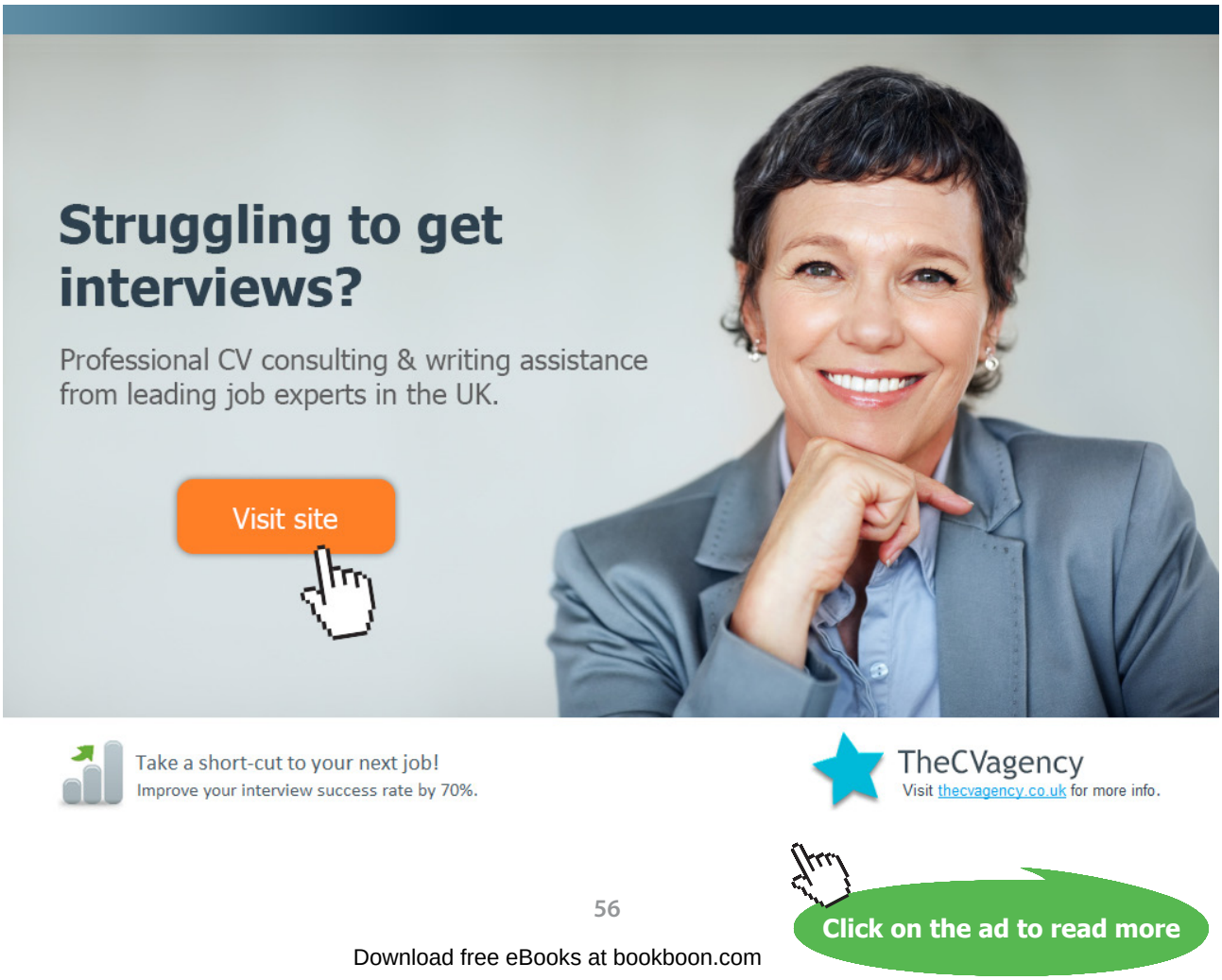
Download free eBooks at bookboon.com

Thus, we have a series of operations (print iNo, print the square of iNo) which is repeated 10 times. A repetition is called a loop. It is illustrated by the following JSP graph:



First the variable iNo is set = 1. Then a loop "Print values until iNo=10" is started. The fact that it is a loop is shown by the subordinate boxes having an asterix in the upper right corner.

The loop consists of three boxes, which in turn print the value of iNo, the value of iNo * iNo (i.e. the square of iNo), and increase the value of iNo by 1. The loop goes on until iNo has reached the value 10.



Download free eBooks at bookboon.com

3.13 The while Loop

Here is a code section that performs the task:

```
iNo = 1;
while (iNo <= 10)
{
   cout << iNo << " " << iNo * iNo << endl;
   iNo++;
}
```

First the variable iNo gets the value 1. Then the loop follows starting with the keyword while, followed by a condition within parenthesis. The operations to be repeated are given within the curly brackets immediately after the while condition.

The while line can be read: ”As long as iNo is less than or equal to 10”. For each turn of the loop iNo and iNo*iNo are printed on the screen, separated by a space and followed by a line break. At the end of the loop iNo is increased by 1.

3.14 The for Loop

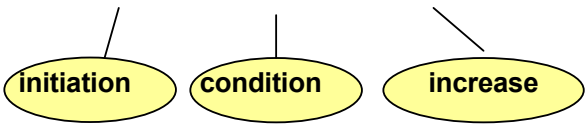
The task was solved by the while loop above. The for loop is another type of loop:

```
for (iNo=1; iNo <= 10; iNo++)
{
   cout << iNo << " " << iNo * iNo << endl;
}
```

This loop does exactly the same thing, namely prints the numbers 1-10 and their squares. The code block however contains only one statement. The actual increase of the iNo value is managed by the parenthesis after the keyword for.

The parenthesis contains three parts, separated by semicolons:

```
for (iNo=1; iNo <= 10; iNo++)
```



The initiation part sets a start value of a variable, often called loop variable, since it controls when to interrupt the loop. The condition part is checked for each turn of the loop. When the condition is false, the loop is interrupted. The increase part changes the value of some variable; mostly it is the loop variable that is increased by 1.

However, you don’t have to start with 1 or increase by 1 for each turn of the loop. The following code example shows how the variable iNo from start is set to 2. The increase part will add 2 for each turn of the loop:

```
for (iNo=2; iNo <= 10; iNo=iNo+2)
{
   cout << iNo << " " << iNo * iNo << endl;
}
```


Download free eBooks at bookboon.com

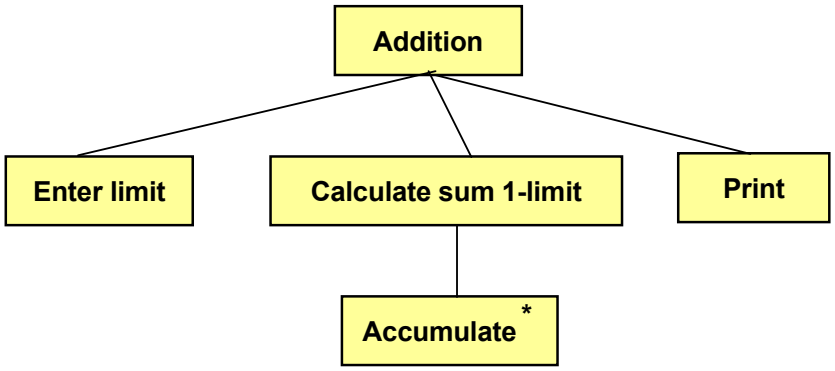
3.14.1 while or for

When should you use the while loop and when the for loop? Many times you can solve the problem with both loop types, and many times it is a question about personal preference. In general, however, if you can predict the number of turns of the loop, the for loop is the best one. If there is an unpredictable situation, e.g. if the loop goes on until the user enters a specific value, or that the random number generator provides a specific number, use the while loop. We will use both alternatives.

3.15 Addition Program

We will create a program that adds the integers 1 + 2 + 3 + 4 + … up to the limit specified by the user. The user should first enter the requested limit. Then we will use a loop that goes from 1 to that limit and sums the numbers. We will then need a variable, which is a kind of accumulater, which stores the sum.

We begin with a JSP graph:



First the user is prompted for a limit. The following loop goes from 1 to limit with the loop variable i. For each turn of the loop we add the value of i to the sum, i.e. we accumulate the numbers. Finally we print the accumulated sum. Note that the operation to be repeated (Accumulate) in the loop is indicated by an asterix in the JSP graph.

```
#include <iostream.h>
void main()
{
  int i, iLimit, iSum = 0;
    cout << "Enter limit: ";
    cin >> iLimit;
    for (i=1; i<=iLimit; i++)
          iSum += i;
    cout << "The sum = " << iSum << endl;
}
```

First, a number of variables are declared. The variable i is used as loop variable, iLimit is used for storage of the user specified limit, and iSum the accumulated sum. Note that, since iSum is increased by a value all the time, it must have a

Download free eBooks at bookboon.com

start value. A declared variable does not automatically get the value 0 or any other value. That is why we must initiate it with 0. The other variables will get fix values during the execution of the program, so they need not be initiated.

The user will then enter a value to be stored in the variable iLimit. The loop then sets a start value = 1 to the loop variable i. The for-condition is that i is not allowed to exceed iLimit. For each turn of the loop the loop variable is increased by 1. That means that the value stated by the user controls the number of turns of the loop.

The repetition code block of the loop contains only one statement. Therefore we don't need any curly brackets surrounding the loop. If, however, the repetition code block contains several statements, they must be surrounded by curly brackets. Compare the if statement, which works in the same way.

The repetition code block contains this statement:

```
iSum += i;
```

which implies that the variable iSum is increased by i for each turn of the loop. This means that the variable iSum will contain the sum 1 + 2 + 3 + …

Finally the value of iSum is printed.

An optional way of writing the for line:

```
for (int i=1; i<=iLimit; i++)
```

Here we declare the variable i inside the for statement. The variable should then not be declared earlier in the program.

Click on the ad to read more

Download free eBooks at bookboon.com

Still another way of coding:

```
for (int i=1; i<=iLimit; iSum+=i++);
```

Here the increase part of the for statement contains the code `iSum+=i++.` Here two things happen, namely that the variable iSum is increased by the value of i, and then i is increased by 1 (i++). This means that we don't need any repetition code block, so we put a semicolon directly after the parenthesis. Consequently the loop consists of one single line.

3.16 Double Loop

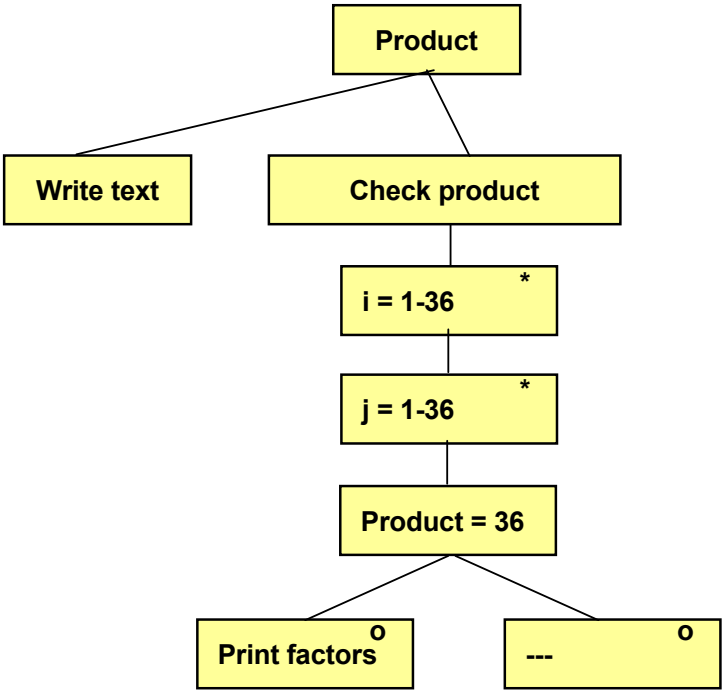
We will now how to use a double loop, i.e. a loop inside another loop. The inner loop will then do all its loop turns for each turn of the outer loop. Here is an example.

We will write a program that figures out all combinations of two integers whose product is 36:

1 x 36
2 x 18
3 x 12
etc.

We let the outer loop control the first factor, which runs from 1 to 36. For each value of the first factor we will go through the values 1-36 for the second factor and check if the product equals 36. If so, the factors are printed.

First we give a JSP graph:



Under "Check product" we have an outer loop with the loop variable i and an inner loop with loop variable j. Inside the loop we check if the product of i and j makes 36. If so, i and j are printed.

Download free eBooks at bookboon.com

Here's the code:

```
#include <iostream.h>
void main()
{
  int i, j;
    cout << "Calculation of produdt" << endl;
    for (i=1; i<=36; i++)
    {
      for (j=1; j<=36; j++)
      {
        if (i*j == 36)
        cout << i << " and " << j << endl;
      }
    }
}
```

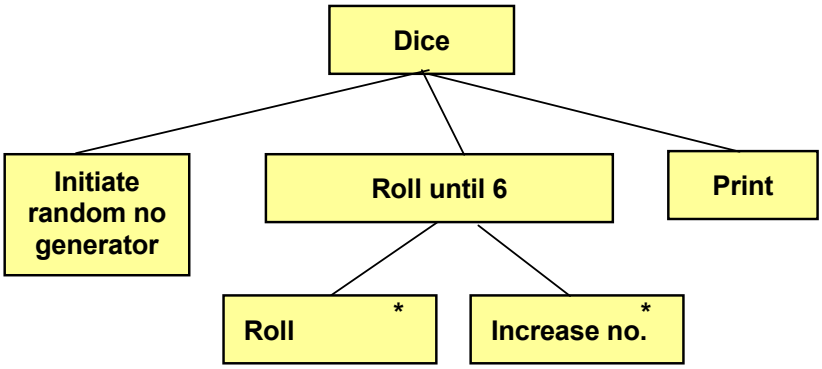
In the double for loop the variable i gets the value 1. The inner loop starts and lets j run through the values 1-36. For each value of j we check if i*j makes 36. If so, we print the values of i and j. When the inner loop has finished, the next turn of the outer loop will start where i is set =2, and the inner loop starts once again and lets j run from 1 to 36.

3.17 Roll Dice

So far we have mainly used the for loop. We will now look at a few situations where the while loop is preferred. We will write a program that rolls a dice until we get 6. Then the number of rolls is printed.

Here we cannot predict how long the loop will run. That depends on the numbers being generated. Therefore, the while loop is perfect.

Let us first create a JSP graph:



We begin with initiating the random number generator to a randomly selected start position. Then the loop is repeated until we get 6. For each turn of the loop we roll the dice once more and increase the number of rolls by 1. When 6 has been achieved, the loop is terminated and the number of rolls is printed. Here's the code:

Download free eBooks at bookboon.com

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
void main()
{
   int iRoll=0, iNoOfRolls=0;
   srand(time(0));
   while (iRoll != 6)
   {
      iRoll = rand()%6+1;
      iNoOfRolls++;
   }
   cout << iNoOfRolls;
}
```

The header file stdlib.h is needed for the random number functions, and time.h is needed for the function time(0) at initiation of the generator.

The variable iRoll is used to store each roll. The reason for initiating it with 0 at the declaration is that it must hold a value when the while loop starts. The value must be something else than 6, otherwise the loop will not start. Any other value will do.

Click on the ad to read more
Download free eBooks at bookboon.com

The variable iNoOfRolls is used to count the number of rolls. It must be initiated to 0, since it is increased by 1 all the time.

The function srand() initiates the generator to a randomly selected start position.

The while loop contains the condition that iRoll must not be 6. As long as no 6 is achieved the loop runs one more turn. For each turn we make another roll stored in iRoll, and the variable iNoOfRolls is increased by 1.

When we get 6, the while condition is false and the loop is terminated. The variable iNoOfRolls then contains the number of rolls, which is printed.

A variant of the program looks like this:

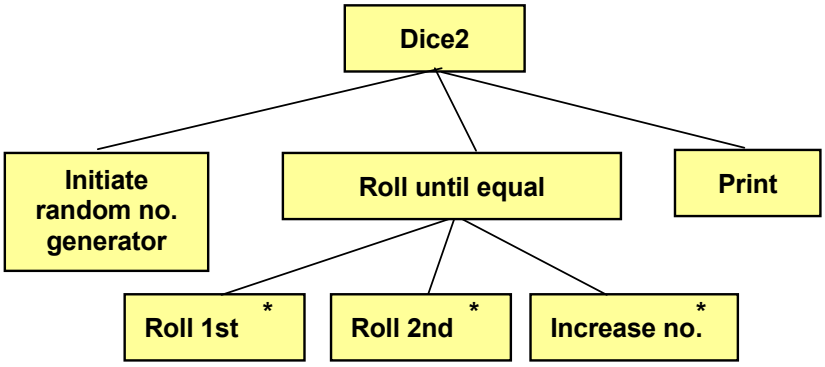
```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
void main()
{
   int iRoll, iNoOfRolls=0;
   srand(time(0));
   do
   {
      iRoll = rand()%6+1;
      iNoOfRolls++;
   } while (iRoll != 6);
   cout << iNoOfRolls;
}
```

The big difference is that the loop has its condition *after* the loop body instead of before. The effect of this is that at least one turn of the loop is executed before the condition is tested. This also means that the variable iRoll needs not be initialized to 0. It will anyway get a new value during the first turn of the loop.

The keyword 'do' is before the loop body, and 'while' followed by the condition right after the ending curly bracket. You must have a semicolon immediately after the condition.

3.18 Two Dice Roll

We will now write a program which repeatedly rolls two dice and checks if the two rolls are equal. When two equal rolls have been achieved, the process is terminated and the number of "double" rolls is printed. We start with a JSP graph:



Download free eBooks at bookboon.com

When having initiated the random number generator to a random start position, the loop begins. For each turn, we roll the 1st and then the 2nd dice, and then increase the counter by 1. When the two rolls are equal the loop is terminated and the number of "double" rolls is printed. Here is the code:

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
void main()
{
  int iRoll1, iRoll2, iCounter=0;
      srand(time(0));
      do
      {
            iRoll1 = rand()%6+1;
            iRoll2 = rand()%6+1;
            iCounter++;
      } while (iRoll1 != iRoll2);
      cout << "The rolls were " << iRoll1 << endl;
      cout << "Number of attempts = " << iCounter << endl;
}
```

The program is similar to the previous with the difference that here we have two variables which store the rolls. Inside the loop iRoll1 and iRoll2 get their values and the number of "double" rolls is increased by 1.

Since the while condition comes after the loop body, at least one loop turn will be executed. The condition is that iRoll1 is not equal to iRoll2. If they are equal the loop is terminated and the dice score and the number of rolls are printed.

3.19 Breaking Entry with Ctrl-Z

We will now use the while loop condition to contain a user input with cin. The program will prompt the user for repeated entry of numbers. The entered numbers are summed. When the user presses Ctrl-Z the entry of numbers is interrupted and their average is printed. Here is the code:

```
#include <iostream.h>
void main()
{
  int iSum=0, i=0, iNo;
      cout << "Enter a number: ";
      while (cin >> iNo)
      {
            iSum += iNo;
            i++;
            cout << "Enter one more number: ";
```


Download free eBooks at bookboon.com

```
        }
        cout << "Average = " << (double)iSum/i << endl;
}
```

The variable iSum is used to store the sum of the entered numbers. The variable i counts the number of numbers.

The while condition contains input of a number from the user. If the input succeeds, the cin function will return a true value. One turn of the loop is then executed. In the loop the variable iSum is increased by the entered value and the variable i, which counts the values, is increased by 1. At the end of the loop the user is prompted for yet another value.

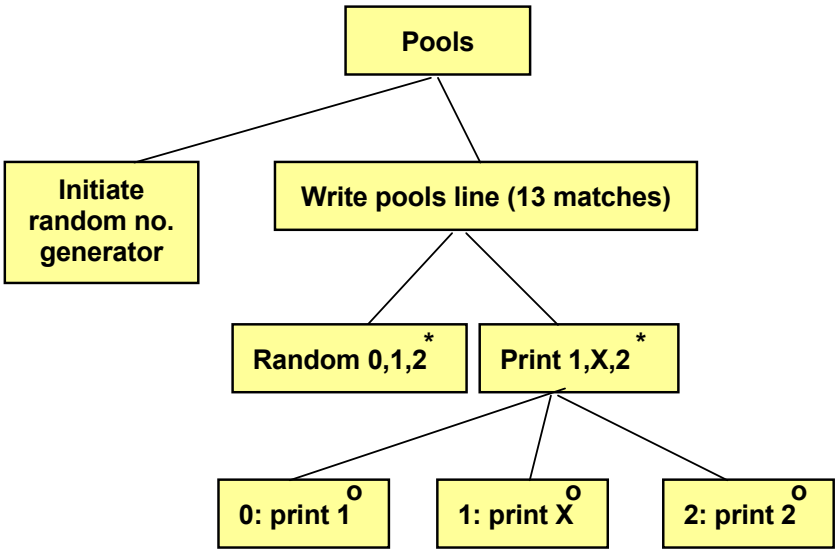
When one turn of the loop has been run, the condition is tested again, i.e. the program halts and waits for a new entry. As long as the user enters numbers, a new loop turn is run. If the user presses Ctrl-Z the function cin returns a false value, which makes the loop to be terminated. Then the average is printed, which is calculated by dividing the sum by the number of values. Since the variable iSum is an integer we must type cast it to double before the division to not loose the decimals.

3.20 Pools

Programming a pools line (1, X or 2) with 13 football matches is another example of how to use the random number generator in a loop. Since we know that a pools line contains 13 matches, we use a for loop. First we create a JSP graph:

Click on the ad to read more

Download free eBooks at bookboon.com



When having intitated the random number generator, the loop begins which should create 1, X or 2 for 13 matches. We do that by create a random number which is 0, 1 or 2. If it is 0, we print '1'. If it was 1, we print 'X'. If it was 2, we print '2'. Here is the code:

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
void main()
{
      int iNo;
      srand(time(0));
      for (int i=1; i <= 13; i++)
      {
            iNo = rand()%3;
            switch (iNo)
            {
            case 0:
                  cout << "1" << endl;
                  break;
            case 1:
                  cout << "   X" << endl;
                  break;
            case 2:
                  cout << "     2" << endl;
                  break;
            }
      }
}
```


Download free eBooks at bookboon.com

When having initiated the random number generator with srand(), the for loop runs from 1 to 13.

For each loop turn we create a random number in the interval 0-2, which is stored in the variable iNo. It is then checked by the switch statement. The different case blocks takes care of the cases 0, 1 and 2. For the value 0, we print '1'. For the value 1, we print 'X' preceded by some blanks which makes the X's appear in a separate column. For the value 2, we print '2' preceded by some more blanks.

3.21 Equation

We will now solve a math problem, namely to solve an equation of 2nd degree. To simplify, we assume that the equation has only integer roots. The equation is:

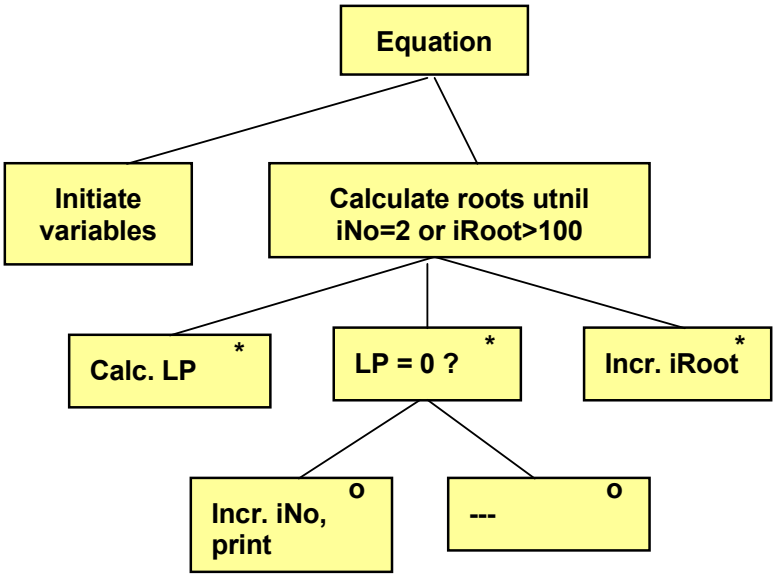
$$x^2 - 6x + 8 = 0$$

Since it is of the 2nd degree it has two roots.

Finding the solution to an equation means to find x values such that the left part (LP) equals the right part (RP), i.e. equal to 0 in our equation.

We create a loop which in turn tests the values 1, 2, 3 … up to 100. The test procedure is to replace x by 1 in LP and calculate if LP equals 0. Then we replace x by 2 and repeat the process until we find two values that match the equation or until 100 has been reached.

First, we create a JSP graph:



The variable iNo is used to count the number of roots to the equation that we have got. The variable iRoot is the x value in the equation which in each loop turn is used to calculate LP. The value of iRoot starts with 1 and is increased by 1 for each loop turn. If the value of LP = 0, we increase the value of iNo and print the root (x value).

Download free eBooks at bookboon.com

Here is the code:

```
#include <iostream.h>
void main()
{
   int iNo=0, iRoot=1, LP;
       while ((iNo<2) && (iRoot<=100))
       {
             LP = iRoot * iRoot - 6 * iRoot + 8;
             if (LP==0)
             {
                   iNo++;
                   cout << iRoot << endl;
             }
             iRoot++;
       }
}
```

First, the value of iNo is set to 0, since it later will be increased by 1 for each found root. The first root value to be tested is 1 (iRoot=1)..

The loop has the condition that iNo should be less than 2, since the number of roots to an equation of 2nd degree is not greater than 2, and iRoot must not exceed 100, since we don't examine roots over 100.,

Rand Merchant Bank uses good business to create a better world, which is one of the reasons that the country's top talent chooses to work at RMB. For more information visit us at www.rmb.co.za

Thinking that can change your world

Rand Merchant Bank is an Authorised Financial Services Provider

Click on the ad to read more

Download free eBooks at bookboon.com

The first statement in the loop body calculates the value of the left part (LP). This is the actual definition of the equation (x^2 - 6x + 8). If this value is = 0, we increase iNo by 1 and the root is printed.

Then we increase the root value by 1.

3.22 Interrupting a Loop - break

Many times you don't want to set an upper limit on the number of loop turns. Then you can use a condition for the while loop which always is true, for example:

```
while (1==1)
```

1 is obviously always equal to 1, so the loop will run an infinite number of turnes. Therefore we need a possibility to, from inside the loop body, interrupt it, i.e. jump out of it and continue with the first statement after the loop. That is accomplished with the keyword:

```
break;
```

We will give a little program example of this. We will write a program where the user repeatedly is prompted for a number, and the program will respond with the square root of the number. Since you cannot calculate the square root of a negative number, we will inside the loop body check whether the user has entered a negative value. If so, the loop is interrupted. Here is the code:

```
#include <math.h>
#include <iostream.h>
void main()
{
  double dNo;
  while (1==1)
  {
    cout << "Enter a number ";
    cin >> dNo;
    if (dNo<=0)
      break;
    cout << "The square root of the number is " << sqrt(dNo)
         << endl;
  }
}
```

To be able to calculate the square root, we must include math.h, which contains code for a large number of math functions.

The while condition is that 1 equals 1, which always is true, i.e. we have created an infinite loop. Inside the loop body the user is first prompted for a number. If the number is less than 0, the loop is interrupted with break. If the number is 0 or positive, the loop goes on with calculating and printing the square root. The function sqrt() is used for this calculation.

Of course you could solve this problem without using an infinite loop, but regard this as an alternative to create loops.

Download free eBooks at bookboon.com

3.23 Summary

In this chapter we have learnt to incorporate intelligence into our programs by means of selections. We have also learnt how if statements can be used to check different situations and perform different tasks depending on the circumstances. We have showed how to combine various conditions in complex situations with the operators && and ||. We have also learnt how to use the modulus operator % and how to perform conditional input of values from the user by placing the input statement with cin inside the if condition.

An alternative to the if statement is the switch statement which is often used in connection with menu programs.

We have in this chapter also introduced loops, which are used to perform a series of operations a repeated number of times. The main loops are the for loop and the while loop.

We have also extended our knowledge about the random number generator, which has been used to roll a dice and play pools game. Finally we have spent some effort by solving mathematical equations by means of loops.

3.24 Exercises

1. Write a program that prompts the user for two values and prints the least of them.
2. Write a program that prompts the user for his age. If he is younger than 15, the text "You'll got to stick to the bike some more time" should be printed. Otherwise the text "You are allowed to drive moped" should be printed.
3. Improve the previous program so that it also pays attention to the driving license age of 18.
4. Write a program that prompts the user for three numbers and prints the greatest of them.
5. Start from the Price Calculation program earlier in this chapter and apply a new discount of 5% if the gross value exceeds 250:- .
6. Continue with the previous program and write code for tax calculation, which is performed so that the user is asked for whether it is food or other products that he has bought. Let the user enter 1 for food and 2 for other products. The program should then add 12% tax for food, or 25% for other products. The tax amount and the final customer price should also be printed.
7. Suppose that the following taxing rules apply:
   a) Income below 10 000:- is not taxable.
   b) For income of 10 000 and more the base tax is always 50%.
   c) For income below 50 000 a tax reduction of 5 000:- is given.
   d) For income over 100 000 there is an extra tax addition of 20% of the portion exceeding 100 000.

   Write a program that prompts the user for his income and calculates the total tax.
8. Write a program that defines whether an entered number is odd or even.
9. Improve the previous program so that it also defines whether the number could be evenly divided by 3.
10. Write a program that prompts the user for how many coins of values 0,50-crowns, 1-crown, 5-crowns and 10-crowns he has in his wallet. The program should then print the total value.
11. Write a program that prompts the user for a price. A discount percent should then be printed according to the following table:

0-100	0%
100-500	5%

Download free eBooks at bookboon.com

500-1000	8%
1000-2000	10%
2000-5000	15%
over 5000	18%

12. Write a program that prompts the user for a quantity and a unit price of a product. If the quantity exceeds 20 and the total price exceeds 1000 kr, the user will get 20% discount. Otherwise, if either the quantity exceeds 20 or the total price exceeds 1000, he will get 10% discount. In all other cases no discount will be given. The total price and the discount should be printed.
13. Use the menu program with the switch statement earlier in this chapter and add the option:
    9. Exit
14. Extend the previous program with the option:
    4. Product
    i.e. the numbers should be multiplied.
15. Write a menu program that prompts the user for three numbers and then displays the following menu:
    1. Least
    2. Greatest
    3. Sum
    The program should also print the requested information.
16. Write a program that prints the numbers 1-10 and their squares.
17. Extend the previous program to also print the cubes of the values.
18. Write a program that prompts the user for integers until the entered number = 0.
19. Extend the previous program so that it also prints the sum of all entered numbers.
20. Write a program that prompts the user for integers and prints a message for each integer whether it is positive or negative. This is repeated until the entered value equals 0.
21. Write a program that prompts the user for integers until the entered value is evenly dividable by 3.
22. In many sports the competitors get scores which are the sum of the scores given by each judge after the highest and lowest score has been deducted. Write a program that prompts for one score from a judge at a time, adds the score and keeps track of the highest and lowest score. The entry is interrupted with Ctrl-Z. Then the competitor's total score should be printed after having deducted the highest and lowest score.
23. Start from the Double Loop program earlier in this chapter, which calculates pair of numbers whose product equals 36. Change it to let the user enter the product to be used.
24. Write a program that calculates the quotient of two numbers. If the quotient = 5, the numbers should be printed. All numbers up to and including 100 should be examined.
25. Start from the Roll Dice earlier in this chapter. Complete it with a printout of all rolls.
26. Change the previous program to roll the dice until it shows 5 or 6.
27. Start from the Two Dice Roll program that rolls two dice at a time. Complete it with a printout of all pair or rolls.
28. Change the previous program to roll the dice until the sum of two rolls is 12.
29. Start from the Pools program earlier in this chapter. Extend it to print 5 lines of pools beside each other, e.g.:

```
1          X          2        X        1
 X          2      1         1          X
1         1          X          2         2
etc.
```


Download free eBooks at bookboon.com

30. Write a program that randomly prints the numbers 0 and 1. It can for instance illustrate tossing of a coin, where 0 and 1 represent the two sides of the coin.
31. Write a program that randomly pulls cards from a pack of cards and prints both colour (spades, diamonds, clubs, hearts) and value (2-10, jack, queen, king, ace). The colour and value of the card should be printed.
32. Start from the equation program earlier in this chapter. Solve the equation:
    $x^2 - 8x + 15 = 0$
33. Write a program that solves the equation of 3rd degree (3 roots):
    $x^3 - 9x^2 + 23x - 15 = 0$

Deloitte.
Discover the truth at www.deloitte.ca/careers
© Deloitte & Touche LLP and affiliated entities.
Click on the ad to read more

Download free eBooks at bookboon.com